

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr. R. Kesavan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of	25%	Applies advanced and relevant	Applies appropriate concepts	Limited application of concepts	Incorrect or no

Engineering Concepts		concepts effectively to solve the problem	with minor errors		application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1. E-commerce Company – 10 Million Product Records

Problem

An e-commerce company needs to store **10 million product records** and retrieve products efficiently.

Solution

I recommend using a **B+ Tree indexed Sorted File organization**.

Design

Product File



Sorted by ProductID



B+ Tree Index



Fast Search

Why B+ Tree?

- Supports fast searching using ProductID.
- Supports range queries such as price between ₹500 and ₹1000.
- B+ Tree keeps index entries organized.
- Suitable for very large databases.

Cost Analysis

For 10 million records, a linear search may require checking up to:

10,000,000 records

B+ Tree search requires approximately:

$O(\log N)$

Therefore, searching is much faster.

Operation	Heap	Sorted + B+ Tree
Insert	Fast	Moderate
Search	Slow	Very fast
Range Search	Slow	Very fast
Storage	Low	Higher

Conclusion

B+ Tree indexed organization is suitable because the company has a very large number of records and needs fast search and range queries.

Q2. Banking System – Transactions by Account Number and Date

Problem

A banking system needs fast retrieval of transactions based on:

- Account number
- Date range

Example:

Account No = 1001

Date = 01-08-2026 to 27-08-2026

Solution

Create a **composite B+ Tree index**:

(AccountNumber, TransactionDate)

Index Structure

B+ Tree Index

↓

AccountNumber

↓

TransactionDate

↓

Transaction Records

Example

Account 1001

|— 01-08-2026

|— 10-08-2026

|— 15-08-2026

|— 25-08-2026

Why?

The first column identifies the account and the second column supports date-range searching.

For example:

SELECT *

FROM Transactions

WHERE AccountNumber = 1001

AND TransactionDate BETWEEN '2026-08-01' AND '2026-08-27';

Conclusion

A **composite B+ Tree index on (AccountNumber, TransactionDate)** provides efficient account-based and date-range retrieval.

Q3. Healthcare System – PatientID and Doctor Name

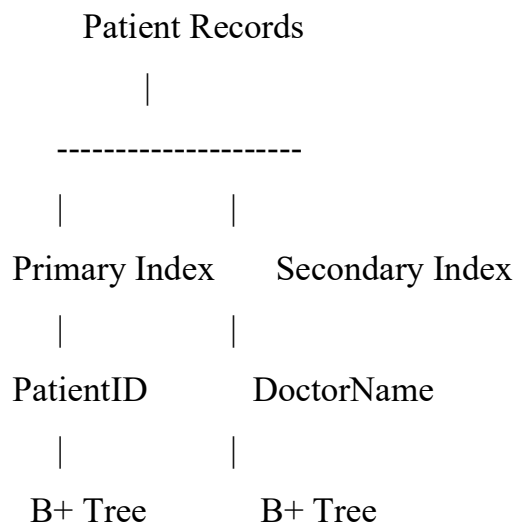
Problem

Patient records must be accessed using:

1. PatientID
2. Doctor name

Solution

Use **multi-level indexing**.



Primary Index

PatientID → Patient Record

Example:

101 → Patient A

102 → Patient B

103 → Patient C

Secondary Index

DoctorName → Patient Records

Example:

Dr. Kumar → 101, 105, 110

Dr. Priya → 102, 108

Why?

- PatientID is unique, so it can be the primary index.
- Doctor name may have many patients, so a secondary index is useful.
- B+ Trees provide efficient searching.

Conclusion

Use a **primary B+ Tree on PatientID** and a **secondary B+ Tree on DoctorName**.

Q4. Logistics Company – B-Tree vs B+ Tree

Problem

A logistics company frequently performs:

- Insertions
- Deletions
- Searches
- Range queries on shipment records

Comparison

Feature	B-Tree	B+ Tree
Search	Fast	Very fast
Insert	Fast	Fast
Delete	Fast	Fast
Range Query	Less efficient	Very efficient
Leaf linking	No	Yes
Data storage	Internal + leaf nodes	Mainly leaf nodes
Database usage	Less common	Very common

B+ Tree Structure

Root

/ \

/ \

Leaf 1 → Leaf 2 → Leaf 3

The leaf nodes are linked.

Why B+ Tree?

Logistics systems may need queries such as:

Find shipments from January to March

B+ Tree can efficiently traverse linked leaf nodes.

Conclusion

B+ Tree is more suitable because it supports efficient insertion, deletion, searching, and especially range queries.

Q5. HR System – 5000 Employees

Problem

An HR system needs to store **5000 employee records** and retrieve employees quickly.

Static Hashing

Use:

Hash function = EmployeeID % number of buckets

For example:

EmployeeID = 105

$105 \% 10 = 5$

Employee 105 is stored in bucket 5.

Static Hashing

Employee ID

↓

Hash Function

↓

Bucket

Advantages:

- Simple
- Fast exact search
- Easy to implement

Disadvantage:

- Fixed number of buckets

- Overflow can occur when the database grows

Extendible Hashing

Extendible hashing uses a directory.

Directory

↓

00 → Bucket

01 → Bucket

10 → Bucket

11 → Bucket

When a bucket overflows:

Overflow

↓

Directory doubles

↓

Bucket splits

Comparison

Feature	Static Hashing	Extendible Hashing
Implementation	Simple	More complex
Growth	Difficult	Easy
Overflow	Common	Handled dynamically
Search	Fast	Fast
Suitable for changing data	Less suitable	More suitable

Conclusion

For 5000 employees with relatively stable data, **static hashing** is sufficient. If employee records are expected to grow significantly, **extendible hashing** is better.

Q6. Airline Reservation System

Problem

The airline needs:

1. Point queries by flight number.
2. Range queries by date.

Example:

Flight number = AI101

and:

Flights from 20-Aug-2026 to 25-Aug-2026

Solution

Use a **combined indexing structure**.

Flight Number → Hash Index

Date → B+ Tree Index

Hash Index

Flight Number



Hash Function



Flight Record

Best for exact searches:

AI101

B+ Tree

Date



B+ Tree



Date Range

Best for:

20-Aug → 25-Aug

Conclusion

Use:

- **Hash index for FlightNumber**
- **B+ Tree index for FlightDate**

This provides fast point and range queries.

Q7. University Student Database

Problem

The university needs fast queries based on:

- Department
- CGPA range

Example:

Department = CSE

CGPA = 8.0 to 9.0

File Organization

Use a **sorted file based on StudentID**.

StudentID



Sorted Student File

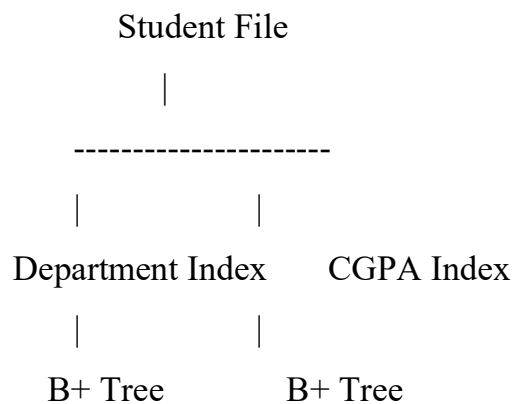
Secondary Indexes

Create:

Department → B+ Tree

CGPA → B+ Tree

Structure:



Example

Department index:

CSE → 101, 105, 110

ECE → 102, 108

IT → 103, 107

CGPA index:

8.0 → Students

8.1 → Students

8.2 → Students

...

Conclusion

A sorted student file with **secondary B+ Tree indexes on Department and CGPA** provides efficient retrieval.

Q8. Disk Block Utilization

Given

Number of records = 100,000

Record size = 50 bytes

Block size = 4 KB

Convert:

4 KB = 4096 bytes

Records per block

$4096 / 50 = 81.92$

Therefore:

81 records per block

Number of blocks

$100000 / 81 = 1234.57$

Therefore approximately:

1235 blocks

Comparison

Organization	Block Utilization	Advantage
Heap	High	Fast insertion
Sorted	High	Fast searching
Hashed	High	Fast exact search

For 81 records:

$81 \times 50 = 4050$ bytes

Unused space:

$4096 - 4050 = 46$ bytes

Utilization:

$4050 / 4096 \times 100$

$\approx 98.88\%$

Conclusion

Approximately **81 records fit in each 4 KB block**, giving about **98.88% block utilization**.

Q9. Social Media – 500 Million Posts

Problem

A social media platform has **500 million posts** and needs searches based on:

- user_id
- timestamp
- hashtag

Solution

Use composite and separate indexes.

Index 1

(user_id, timestamp)

Useful for:

Find user's posts

during a specific time period

Index 2

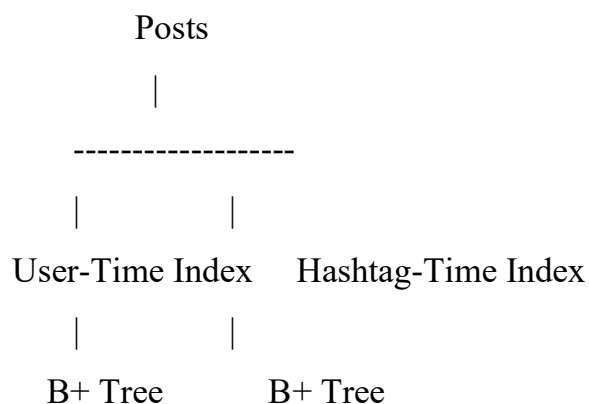
(hashtag, timestamp)

Useful for:

Find posts with #AI

during a specific date range

Structure



Example

Query:

user_id = 101

timestamp = August 2026

Use:

(user_id, timestamp)

Query:

hashtag = #AI

timestamp = August 2026

Use:

(hashtag, timestamp)

Conclusion

For 500 million records, **composite B+ Tree indexes** provide efficient searching while avoiding a single huge index for every query type.

Q10. Supermarket Billing System – 1 Million Transactions

Problem

A supermarket generates **1 million transactions per day**.

We compare:

- Heap File
- Sorted File
- Hashed File

Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	Very Fast	Slow	Fast
Delete	Moderate	Moderate	Fast
Search	Slow	Fast	Very Fast
Range Search	Slow	Very Fast	Poor
Organization	Unordered	Sorted	Hash buckets

Heap File

Records are inserted wherever space is available.

Insert → Very fast

Search → Slow

Suitable when frequent insertion is required.

Sorted File

Records are maintained in sorted order.

Search → Fast

Range Query → Very Fast

Insert → Slower

Hashed File

Hash function determines the location.

TransactionID



Hash Function



Bucket

Exact transaction searches are very fast.

Final Recommendation

For a supermarket billing system, **hashed file organization** is best for exact transaction lookup because billing systems commonly search using a unique transaction/bill ID.

If the system frequently performs **date or amount range reports**, a **sorted file with B+ Tree indexing** would be better.